

AID DL 스터디 · 밑바닥부터 시작하는 딥러닝

1장 헬로 파이썬

AI 시대에 파이썬을 읽는 법

책 25-45쪽 리뷰 · 파이썬이 AI에서 쓰이는 곳 · 바이브 코딩 활용

2026. 10

1장은 딥러닝을 직접 구현하기 위한 준비 장이에요

책에서 쓰는 도구는 **파이썬 3 · NumPy · matplotlib** 세 가지뿐이에요. 오늘은 문법 리뷰에 더해 "왜 지금 AI에서 파이썬인가"를 함께 봐요.

01 AI 시대에 파이썬을 알아야 하는 이유

02 AI 개발 단계별 파이썬의 쓰임

03 VS Code 실습 환경

04 1.3-1.4 기본 문법 · 클래스

05 1.5 넘파이 · 브로드캐스트 (가장 중요)

06 1.6 matplotlib · 이미지 = 배열

07 더 알면 좋은 파이썬 · 바이브 코딩

08 확인 문제 · 실습 과제

사람이 설계하는 부분은 파이썬, 무거운 계산은 C++/CUDA

작은 모델이든 대형 LLM이든 딥러닝 코드는 2층 구조예요.

위층 · 파이썬

모델 구조 설계, 데이터 준비, 학습 반복, 실험 관리

$y = x @ W + b$ · `class MyModel(nn.Module)`

▼ 계산을 맡김

아래층 · C/C++, CUDA

행렬 곱셈 같은 대량 계산을 CPU·GPU에서 실행
넘파이 내부, PyTorch 내부 엔진(ATen)

책 p.41 NOTE: 파이썬은 느리지만 넘파이는 내부를 C/C++로 처리해서 **편한 문법 + 빠른 속도**를 함께 얻어요. 책 p.26의 "프레임워크가 파이썬용 API를 제공한다"도 같은 뜻이에요.

작은 모델부터 대형 LLM까지, 시작과 조립은 파이썬

"파이썬이 아니면 못 만든다"가 아니라 "거의 모두가 파이썬으로 시작하고 조립한다"가 정확해요.

규모	파이썬의 역할	예시
교육·기초 모델	전부 파이썬	이 책(NumPy), GPT-2를 직접 만드는 nanoGPT(PyTorch)
대형 LLM 학습	모델 정의·학습 코드는 파이썬, 계산 커널은 C++/CUDA	PyTorch, JAX
공개 모델 활용	모델을 받고, 불러오고, 파인튜닝	Hugging Face Transformers
예외	최고 성능이 필요한 곳은 파이썬 없이 만들기도 해요	llama.cpp(C/C++), llm.c(C·CUDA로 GPT-2 학습)

파이썬이 AI의 표준이 된 세 가지 이유 (책 p.25-26)

① 배우기 쉬워요

문법이 영어와 비슷해서 연구자가 수학 아이디어를 빠르게 코드로 옮길 수 있어요.

② 수치 계산 도구가 먼저 쌓였어요

넘파이·SciPy 위에 딥러닝 프레임워크가 만들어졌어요.

③ 생태계 효과

논문 코드, 튜토리얼, 공개 모델이 전부 파이썬이라 새로 오는 사람도 파이썬을 써요.

GitHub Octoverse 2025: 전체 기여자 수 1위는 TypeScript였지만, AI 분야 저장소에서는 파이썬이 가장 많이 쓰였어요.

우리에게 의미하는 것 — 책·논문·AI가 생성한 딥러닝 코드가 모두 파이썬이에요. 직접 다 쓰지 못해도 읽고 판단할 수 있어야 해요. C++·CUDA는 지금 몰라도 괜찮아요.

AI 개발 7단계, 모든 단계의 기본 도구가 파이썬

단계	하는 일	대표 도구	쓰이는 1장 문법
1. 데이터 준비	읽기·정리·나누기	pandas, NumPy	리스트, 슬라이싱, 불리언 인덱싱
2. 모델 설계	층을 쌓아 구조 정의	PyTorch	클래스, <code>__init__</code> , <code>self</code>
3. 학습	데이터를 돌며 가중치 수정	PyTorch, DataLoader	for 문, 함수, 배열 연산, shape
4. 평가·시각화	정확도·손실 확인	matplotlib	<code>plt.plot</code> , 딕셔너리
5. 공개 모델 활용	받아 쓰기·파인튜닝	Hugging Face Transformers	딕셔너리(설정·가중치), dtype
6. 서빙·데모	웹·앱에서 쓰게 만들기	FastAPI, Gradio, vLLM	함수, 딕셔너리(JSON)
7. LLM API·에이전트	Claude·GPT 호출, 도구 연결	Anthropic SDK, MCP SDK	리스트·딕셔너리, 함수, if

출처: pandas · PyTorch DataLoader · FastAPI · Gradio · vLLM · Anthropic SDK · MCP Python SDK

1장 문법만으로 LLM API 코드를 읽을 수 있어요

```
import anthropic

client = anthropic.Anthropic()      # 클래스 → 인스턴스 (1.4.2)

messages = [                        # 리스트 안에 딕셔너리
    {"role": "user",                # (1.3.4, 1.3.5)
     "content": "퍼셉트론을 설명해줘"}
]

response = client.messages.create(  # 메서드 호출
    model="claude-...",
    max_tokens=300,
    messages=messages,
)
print(response.content[0].text)    # 인덱싱 + 속성 접근
```

새 문법은 0개

클래스, 리스트, 딕셔너리, 인덱싱만으로 AI 서비스 코드의 뼈대가 읽혀요.

딕셔너리가 핵심

API 요청·응답(JSON), 모델 설정, 가중치 저장 (`state_dict`)이 모두 딕셔너리 구조예요. 책 3장에서도 `network['w1']` 처럼 써요.

파이썬 3.13 + VS Code면 아나콘다 없이도 충분해요

환경 준비 (처음 한 번)

1. 확장 설치: Microsoft **Python**, **Jupyter**
2. 스터디 폴더 열기
3. `Ctrl+Shift+P` → Python: Create Environment → Venv
4. 터미널에 `(.venv)` 확인 후
`pip install numpy matplotlib ipykernel`
5. Python: Select Interpreter → `.venv`
6. 책 예제: `git clone WegraLee/deep-learning-from-scratch`

실행 방법 3가지

방법	책과 대응	용도
터미널 <code>python</code>	1.3 대화 모드	한 줄 실험
<code>.py</code> + ▶	1.4 스크립트	완성 코드
<code>.ipynb</code> 셀	1.5·1.6	공부용 추천

경로 오류 예방: 설정에서 `python.terminal.executeInFileDir` 를 켜면 책처럼 파일 위치 기준으로 실행돼요.

기본 문법은 그대로 딥러닝 코드의 부품이 돼요

문법	핵심	딥러닝에서
산술 연산	<code>7/5 = 1.4</code> , <code>7//5 = 1</code> , <code>3**2 = 9</code>	<code>**</code> 는 오차 제곱(4장 손실 함수)
자료형·변수	동적 언어: 자료형을 파이썬이 정해요. 정수 $\times$ 실수 = 실수	실행해야 타입 오류가 보이는 이유
리스트	인덱스는 0부터, <code>a[0:2]</code> 는 끝 번호 미포함	미니배치 자르기 <code>x[i:i+100]</code>
딕셔너리	키와 값의 쌍 <code>{'height': 180}</code>	가중치 <code>network['W1']</code> , JSON
bool	<code>and</code> , <code>or</code> , <code>not</code>	2장 AND·OR 게이트와 같은 논리
if · for	들여쓰기(공백 4칸)로 블록 구분	for 문 = 4장 학습 루프의 골격
함수	<code>def hello(object):</code>	<code>def AND(x1, x2)</code> , <code>def sigmoid(x)</code>

책의 **Man** 클래스와 PyTorch 모델은 같은 구조예요

책 (1.4.2)

```
class Man:
    def __init__(self, name):    # 생성자
        self.name = name       # 속성

    def hello(self):            # 메서드
        print("Hello " + self.name)

m = Man("David")                # 인스턴스
m.hello()
```

PyTorch 모델

```
class MyModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc = nn.Linear(2, 1) # 가중치

    def forward(self, x):          # 계산
        return self.fc(x)

model = MyModel()
y = model(x)
```

`self.name` → `self.fc` (가중치를 가진 층), `hello()` → `forward()`. 신경망의 층은 **가중치(속성) + 계산 방법(메서드)**을 함께 가지므로 클래스가 딱 맞아요. 책 5장부터 `class Affine`, `class Relu` 로 이어져요.

딥러닝 계산은 거의 전부 배열 연산이에요

```
import numpy as np
x = np.array([1.0, 2.0, 3.0])
y = np.array([2.0, 4.0, 6.0])
x + y      # [3. 6. 9.] 원소별
x * y      # [2. 8. 18.] 원소별 곱
```

```
A = np.array([[1, 2], [3, 4]])
A.shape      # (2, 2) 형상
A.dtype      # int64 자료형
A * A        # 원소별 곱 (행렬곱 아님!)
A @ A        # 행렬곱 (3장)
```

리스트 `[1,2]+[3,4] = [1,2,3,4]` (이어 붙이기)
 넘파이 배열 = `[4, 6]` (덧셈)

차원	이름	딥러닝 예시 (shape)
1	벡터	편향 b (50,)
2	행렬	가중치 W (784, 50)
3	텐서	LLM 입력 (배치, 토큰 수, 임베딩)
4	텐서	이미지 묶음 (배치, 채널, 높이, 너비)

dtype = 메모리: float16은 숫자 하나에 2바이트. 7B 모델을 16비트로 올리면 약 14GB, 4비트 양자화하면 약 3.5GB예요.

모양이 달라도 작은 쪽을 늘려서 계산해요

규칙: shape을 맨 뒤 차원부터 비교해서, 크기가 같거나 한쪽이 1이면 계산돼요. PyTorch도 같은 규칙이에요.

연산	결과 shape	설명
$(2, 2) \times \text{스칼라 } 10$	$(2, 2)$	책 그림 1-1
$(2, 2) \times (2,)$	$(2, 2)$	책 그림 1-2: <code>[10, 20]</code> 이 두 줄로 늘어나요
$(100, 50) + (50,)$	$(100, 50)$	신경망에서 편향 b 를 더하는 순간 <code>x @ W + b</code> (3장)
$(3,) + (3, 1)$	$(3, 3)$	오류 없이 엉뚱한 결과 — 가장 흔한 숨은 버그
$(2, 3) + (2,)$	오류	맨 뒤 차원 3과 2가 달라서 불가

바이브 코딩 팁: AI에게 "각 단계마다 `print(x.shape)` 또는 `assert` 를 넣어줘"라고 요청하세요.

같은 계산, for 문 대신 넘파이로 하면 약 500배 빨라요

```
# 방법 1: for 문 3중 반복 → 약 1.4초
for i in range(100):
    for j in range(50):
        s = 0.0
        for k in range(784):
            s += x[i, k] * W[k, j]
        out[i][j] = s + b[j]

# 방법 2: 넘파이 한 줄 → 약 0.003초
y = x @ W + b
```

≈ 500 ×

신경망 한 층 ($100 \times 784 \rightarrow 50$)
결과는 완전히 같음 · 직접 측정

속도(C/C++ 내부 처리)와 가독성($Y = XW + B$ 수식이 그대로 코드) 두 가지 이유예요. 같은 원리가 GPU에서 더 크게 작동하는 것이 PyTorch예요.

컴퓨터에게 이미지는 숫자 배열이에요

```
X = np.array([[51, 55], [14, 19], [0, 4]])
X = X.flatten() # [51 55 14 19 0 4]
X[X > 15] # [51 55 19] 불리언 인덱싱
```

```
import matplotlib.pyplot as plt
x = np.arange(0, 6, 0.1)
plt.plot(x, np.sin(x), label="sin")
plt.legend(); plt.show()
```

```
img = imread('lena.png')
img.shape # (높이, 너비, 채널)
```

1장 기능	뒤에서 쓰이는 곳
<code>flatten()</code>	MNIST $28 \times 28 \rightarrow 784$ 개로 펼치기 (3장)
<code>X[X > 15]</code>	계단 함수·ReLU 구현 (3·5장)
<code>np.sin(x)</code>	<code>np.exp(-x)</code> 로 시그모이드 (3장)
<code>plt.plot</code> , <code>legend</code>	손실 곡선, 훈련 vs 시험 정확도 (4·6장)
<code>imread</code> , <code>imshow</code>	이미지 = 배열, 비전 AI의 출발점

AI가 짠 코드에 자주 나오는 문법, "읽을 수 있는 수준"이면 충분해요

문법	생김새	AI 코드에서 쓰이는 곳
f-string	<code>f"epoch {i}: loss {loss:.4f}"</code>	학습 진행 출력, 프롬프트에 변수 넣기
리스트 컴프리헨션	<code>[len(s) for s in texts]</code>	데이터 전처리 한 줄
타입 힌트	<code>def forward(self, x: Tensor) -> Tensor:</code>	입출력 형태 메모, AI 코드에 거의 항상 등장
with 문	<code>with torch.no_grad():</code>	평가 시 미분 끄기, 파일 자동 닫기
데코레이터	<code>@app.get("/") , @mcp.tool()</code>	웹 API·AI 도구 등록
이터레이터	<code>for x, t in loader:</code>	배치 단위 데이터, LLM 스트리밍 응답
<code>**kwargs</code>	<code>model(**inputs)</code>	딕셔너리를 인수로 풀기 (Hugging Face)
async / await	<code>await client.messages.create(...)</code>	여러 LLM API 동시 호출

추천 순서 f-string → 리스트 컴프리헨션 → 타입 힌트 → with → 데코레이터

출처: Python typing · torch.no_grad · FastAPI · MCP Python SDK

AI가 쓰고 사람이 검토하는 방식에 잘 맞지만, 약점도 알아야 해요

이점

- **AI가 잘 쓰는 언어:** 널리 쓰이는 언어일수록 코드 생성 성능이 높았어요 (MultiPL-E)
- **검토가 쉬움:** 짧고 영어에 가까운 문법
- **바로 실행:** 컴파일 없이, 노트북 셀 단위로 확인
- **라이브러리:** `pip install` 한 줄로 기능 추가
- **AI 서비스 연결:** LLM API·MCP가 파이썬 SDK 제공

주의점 → 대처

- **실행해야 오류가 보임** → 타입 힌트, `assert` 요청
- **for 문은 느림** → "벡터 연산으로 바꿔줘"
- **버전 충돌·오래된 API** → 버전 명시, `requirements.txt`
- **들여쓰기 깨짐** → `Shift+Alt+F` 자동 정리

AI에게 이렇게 요청하면 실패가 줄어요

1. 환경을 먼저 알려주기

"Windows, VS Code, Python 3.13, NumPy만 사용" — 바로 실행되는 코드가 나올 확률이 높아져요.

2. 작게 나눠 요청하고 바로 실행하기

"데이터 불러오기"만 먼저 만들고 실행해 본 뒤 다음 단계로 넘어가요.

3. 오류는 메시지 전체를 붙여넣기

마지막 줄(`ValueError: ...`)과 줄 번호가 가장 중요해요.

4. 설명과 검증 코드를 함께 요청하기

"각 줄에 주석 + 책의 어느 부분인지 알려줘", "shape 출력과 작은 테스트도 넣어줘"

스터디 확인 문제

1. 파이썬 3에서 `7 / 5` 와 `7 // 5` 의 결과는?
2. `a = [1, 2, 3, 4, 99]` 일 때 `a[1:-1]` 의 결과는?
3. `[1, 2] + [3, 4]` 와 `np.array([1, 2]) + np.array([3, 4])` 의 결과는 각각?
4. `A = np.array([[1, 2], [3, 4]])` 일 때 `A * A` 와 `A @ A` 는 각각?
5. 결과 shape은? ① (4, 3) + (3,) ② (4, 3) + (4,) ③ (5, 1) + (1, 6)
6. 클래스에서 `__init__` 과 `self` 의 역할을 한 문장으로 설명해 보세요.
7. 파라미터 80억(8B) 개 모델을 float16으로 저장하면 대략 몇 GB?

실습 과제 — ① `man.py` 에 `introduce(self, age)` 메서드 추가 ② `x = np.arange(-5, 5, 0.1)` 로 `(x > 0).astype(int)` 그래프 그리기 (3장 계단 함수 예고편) ③ 이미지를 `imread` 로 읽고 `shape`, `dtype` 출력 ④ 실습 코드 하나를 AI로 PyTorch 변환 후 대응 관계 설명하기

정답

#	정답
1	1.4 , 1
2	[2, 3, 4]
3	[1, 2, 3, 4] (이어 붙이기), [4, 6] (원소별 덧셈)
4	$A * A = [[1, 4], [9, 16]]$ (원소별), $A @ A = [[7, 10], [15, 22]]$ (행렬곱)
5	① (4, 3) ② 오류 (맨 뒤 3과 4가 다름) ③ (5, 6)
6	<code>__init__</code> 은 인스턴스를 만들 때 한 번 실행되는 초기화 메서드, <code>self</code> 는 그 인스턴스 자신
7	$8B \times 2\text{바이트} = \text{약 } 16GB$

다음 시간: **2장 퍼셉트론** — 1장의 `and` · `or` 와 원소별 곱셈 `np.sum(w * x) + b` 가 그대로 나와요.